BUG-23 — Objects with field-level security but no object-level permission are dropped from object views

Permission Explorer — root cause & fix

2026-06-15

BUG-23 — FLS-only objects are missing from object-centric views

Summary

Permission Explorer builds every object-centric view from `ObjectPermissions` alone. When a grantor has `FieldPermissions` (field-level security) on an object but **no** `ObjectPermissions` row for that object, the object is silently dropped: it never appears in the grantor's Objects list, it is not counted, and it does not appear as a grantor in the object reverse-lookup. The field grants themselves still render in the Fields sections, so the data is loaded — it is only the *object* dimension that loses it.

This was reported against the **Bullhorn Community Profile** (a Guest profile): `OrderItem` has field grants but shows "No object permissions for this grantor" and cannot be found in the profile's Objects list. The pattern is general, not specific to OrderItem or to this profile.

Reproduction and confirming data

The Bullhorn Community Profile's permissions hang off its implicit, profile-owned PermissionSet `0PS0P000000b5cDWAQ`. A live query confirms three field grants on `OrderItem` and zero object grants:

Grantor (implicit PS)	Object	Field	Read	Edit	ObjectPermissions row?
0PS0P000000b5cDWAQ	OrderItem	OrderItem.Description	Yes	Yes	No
0PS0P000000b5cDWAQ	OrderItem	OrderItem.ListPrice	Yes	No	No
0PS0P000000b5cDWAQ	OrderItem	OrderItem.ProductCode	Yes	No	No

`SELECT … FROM ObjectPermissions WHERE SobjectType = 'OrderItem' AND ParentId = '0PS0P000000b5cDWAQ'` returns zero rows, while the matching `FieldPermissions` query returns the three rows above. This is a normal Salesforce state: field-level security can be set on an object whose object-level CRUD is not granted in the same parent (object access may come from elsewhere, or FLS was configured without CRUD).

Root cause

Three code paths derive objects exclusively from the `ObjectPermissions`-backed index `objPermsByParent` / `objPermsBySobj`, and never consult `fieldPermsByParent` / `fieldPermsBySobj`:

The grantor detail (Profile and PermSet) Objects table, in `ParentDetailSubSections`, set `objRows` from `objPermsByParent` only. The per-user effective object map, `effectiveObjects()`, iterated only `objPermsByParent`, so a user with FLS-only access to an object never saw that object in their effective view. The object reverse-lookup, `ObjectDetail`, built its Grantors table and "Grantors" count from `objPermsBySobj` only, so a profile that grants an object purely through FLS was absent from that object's grantor list.

Because the field grants do flow through the separate field index, the Fields sections and the "Fields granted" counts remained correct throughout — which is why the loss is easy to miss and presents as a confusing "the object isn't there, but its fields are."

Scope

This is widespread. Org-wide, `FieldPermissions` reference **746 distinct objects** while `ObjectPermissions` reference **1,179**; any grantor holding FLS on an object without a co-located object-permission row triggers the gap. OrderItem on the Guest profile is one visible instance among many, spanning both profiles and permission sets.

Fix

The fix introduces a single shared notion of a **field-only object** — an object a grantor reaches through FLS with no object-level CRUD row — surfaced consistently across all three views, flagged with an `FLS-only` pill so admins can tell it apart from a true object grant.

A new helper, `fieldOnlyObjectRows(objPermRows, fieldRows)`, takes a grantor's already-collected object-permission rows and field-permission rows and returns synthetic object rows (CRUD all false, `fieldOnly: true`, with a `fieldGrantCount`) for the SObjectTypes that appear only in field permissions. `ParentDetailSubSections` now appends these to `objRows`, so the Objects table for any Profile or PermSet includes FLS-only objects, badged `FLS-only (n)`. `effectiveObjects()` now performs an equivalent pass over each non-muting source's field permissions, adding any object not already present as a `fieldOnly` entry with all-false CRUD; the user detail view's "Effective objects" list and count therefore include them, and they obey the existing "hide zero-grant objects" toggle. `ObjectDetail` now unions FLS-only grantors into its Grantors table and "Grantors" count, badging them the same way.

The change is display-completing rather than semantics-changing. The "Objects granted" / "Fields granted" stat cards on the Profile and PermSet headers still count true object-level and field-level grants respectively, so "granted" keeps its object-level meaning; FLS-only objects are additive in the tables and clearly labeled. Critically, `userEffectiveSignature` only emits object (`O:`) tokens for truthy CRUD flags, so the synthetic all-false rows added to `effectiveObjects()` produce no spurious entries in Compare Users or Change Impact diffs.

Validation

The edited `PermissionExplorer.jsx` bundles cleanly with esbuild (jsx loader, react marked external) with zero errors or warnings. The SOQL evidence above confirms the OrderItem case the fix targets. After the fix, opening the Bullhorn Community Profile lists `OrderItem` in its Objects table with an `FLS-only (3)` badge and dashes across CRUD; opening `OrderItem` lists the Bullhorn Community Profile among its grantors with the same badge.

Notes for release

This is drafted as **BUG-23** against the current `v3.7.0` and is suggested for a `v3.7.1` patch. The code comments, the changelog/version-history block at the top of `PermissionExplorer.jsx`, the `package.json` version, and any requirements-spec section are intentionally left for a release pass; only the functional code has been changed so the diff stays easy to review.